

Parallel and Distributed IR

9.1 Introduction

As document collections grow, the cost of managing them with traditional IR systems increases.

Searching (query processing) and indexing (creating data structures to enable fast searches) take more time and resources as the collection size grows. Larger collections lead to longer response times, which can make the system unusable if performance degrades too much.

Parallel IR uses multiple processors to speed up tasks, while distributed IR spreads data across computers. These approaches improve speed and scalability for modern search systems, like web search engines, which rely on high query rates for revenue.

9.1.1 Parallel Computing

Parallel computing is the simultaneous application of multiple processors to solve a single problem, where each processor works on a different part of the problem.

With parallel computing, the overall time required to solve the problem can be reduced to the amount of time required by the longest running part.

As long as the problem can be further decomposed into more parts that will run in parallel, we can add more processors to the system, reduce the time required to solve the problem, and scale up to larger problems.

Processors can be combined in a variety of ways to form parallel architectures. Flynn has defined a commonly used taxonomy of parallel architectures based on the number of the instruction and data streams in the architecture. The taxonomy includes four classes:

- SISD single instruction stream, single data stream
- SIMD single instruction stream, multiple data stream
- MISD multiple instruction stream, single data stream
- MIMD multiple instruction stream, multiple data stream.

The **SISD** class includes the traditional von Neumann computer running sequential programs, e.g., uniprocessor personal computers.

SIMD computers consist of N processors operating on N data streams, with each processor executing the same instruction at the same time.

MISD computers use N processors operating on a single data stream in shared memory. Each processor executes its own instruction stream, such that multiple operations are performed simultaneously on the same data item. MISD architectures are relatively rare. **Systolic arrays** are the best known example.

MIMD is the most general and most popular class of parallel architectures. A MIMD computer contains N processors, N instruction streams, and N data streams. The processors can work on separate, unrelated tasks, or they can cooperate to solve a single task, providing a great deal of flexibility. MIMD systems with a high degree of processor interaction are called *tightly coupled*, while systems with a low degree of processor interaction are *loosely coupled*. Examples of MIMD systems include multiprocessor PC servers, symmetric multiprocessors (SMPs) such as the Sun HPC Server, and scalable parallel processors such as the IBM SP2.

Although MIMD typically refers to a single, self-contained parallel computer using two or more of the same kind of processor, MIMD also characterizes distributed computing architectures. In distributed computing, multiple computers connected by a local or wide area network cooperate to solve a single problem. Even though the coupling

between processors is very loose in a distributed computing environment, the basic components of the MIMD architecture remain. Each computer contains a processor, control unit, and local memory, and the local or wide area network forms the communication network between the processors.

The main difference between a MIMD parallel computer and a distributed computing environment is the cost of interprocessor communication, which is considerably higher in a distributed computing environment. As such, distributed programs are usually coarse grained, while programs running on a single parallel computer tend to be finer trained. Granularity refers to the amount of computation relative to the amount of communication performed by the program. Coarse-grained programs perform large amounts of computation relative to communication; fine-grained programs perform large amounts of communication relative to computation. Of course, an application may use different levels of granularity at different times to solve a given problem.

9.1.2 Performance Measures

A number of metrics are available to measure the performance of a parallel algorithm.

One such measure is the speedup obtained with the parallel algorithm relative to the best available sequential algorithm for solving the same problem, defined as:

$$S = \frac{\text{Running time of best available sequential algorithm}}{\text{Running time of parallel algorithm}}$$

Ideally, when running a parallel algorithm on N processors, we would obtain perfect speedup, or $S = N$. In practice, perfect speedup is unattainable either because the problem cannot be decomposed into N equal subtasks, the parallel architecture imposes control overheads (e.g., scheduling or synchronization), or the problem contains an inherently sequential component.

Amdahl's law states that the maximal speedup obtainable for a given problem is related to f , the fraction of the problem that must be computed sequentially. The relationship is given by:

$$S \leq \frac{1}{f + (1 - f)/N} \leq \frac{1}{f}$$

Another measure of parallel algorithm performance is efficiency, given by:

$$\phi = \frac{S}{N}$$

where S is speedup and N is the number of processors.

Ideal efficiency occurs when $Q = 1$ and no processor is ever idle or performs unnecessary work. As with perfect speedup, ideal efficiency is unattainable in practice.

Ultimately, the performance improvement of a parallel program over a sequential program should be viewed in terms of the reduction in real time required to complete the processing task combined with the additional monetary cost associated with the parallel hardware required to run the parallel program.

9.2 Parallel IR

9.2.1 Introduction

We can approach the development of parallel information retrieval algorithms from **two different directions**.

One possibility is to **develop new retrieval strategies that directly lend themselves to parallel implementation**.

The other possibility is to **adapt existing, well studied information retrieval algorithms to parallel processing**. The modifications required to adapt an existing algorithm to parallel implementation depend on the target parallel platform.

We will investigate techniques for applying a number of retrieval algorithms to both MIMD and SIMD architectures.

9.2.2 MIMD Architectures

IR on MIMD Systems: Two Main Approaches

There are two primary ways to leverage MIMD systems for IR:

2.1. **Multitasking (Non-Cooperative Parallelism)**

- Each **processor runs an independent search engine**.
- A **broker distributes incoming queries** to available processors.
- **Search engines do not share computation for a single query**.
- **Shared resources** may include:
 - **Code libraries**
 - **File system caches**
 - **Shared memory**

Advantages:

- **Simple** to implement
- **Scales easily** (add more processors = handle more queries)

Limitations:

- **Individual query response time remains unchanged**
- **Potential I/O bottlenecks due to shared disk access**

Optimization Tips:

- Increase the number of disks and I/O channels
- Distribute or replicate index data across disks
- Use RAID (Redundant Array of Independent Disks) for better throughput

2.2. Cooperative Parallelism (Query-Level Parallelism)

- Multiple processors collaborate to process a single query.
- The broker:
 - Splits a query into subtasks
 - Sends subtasks to different processors
- Each processor evaluates part of the query and returns partial results
- The broker merges these into the final result

Key Benefit: Significantly improves query response time

3. Data Partitioning Strategies

The effectiveness of parallel IR depends on how data and computation are partitioned across processors.

3.1. Document Partitioning

- The document collection is horizontally partitioned.
- Each processor gets a subset of documents.
- Every processor runs the entire query on its subset.

During Query:

- Each processor computes document scores independently.
- The broker merges partial results into the final ranked list.

3.2. Term Partitioning

- The inverted index is vertically partitioned by terms.
- Each processor handles a subset of index terms.

During Query:

- The query is broken into individual terms.
- Each term is sent to the processor storing its inverted list.
- Each processor returns partial document scores.
- The broker aggregates these results.

Performance Consideration:

- Skewed term distribution → Document partitioning is better
- Uniform term usage in queries → Term partitioning performs better

4. Inverted File Indexing in Parallel IR

4.1. Logical Document Partitioning

- A global inverted file is shared.
- Dictionary entries include P pointers (P = number of processors).
- Each pointer leads to the section of the inverted list relevant to that processor's documents.

Query Processing:

- Dictionary and index are loaded into shared memory.
- All processors read from this shared data without lock contention.
- Document score accumulators are shared but partitioned → no conflicts.

Index Construction:

- Each processor indexes a subset of documents.
- All inverted lists are merged using a heap-based priority queue.

4.2. Physical Document Partitioning

- Each processor builds a self-contained index for its own documents.
- No shared index data during query processing.

Query Processing:

- Broker sends the query to all processors.
- Each processor returns a local hit-list.
- Broker merges them using a priority queue.

Score Consistency:

- Global term statistics must be used to ensure consistent ranking.
- Two approaches:
 1. Store global stats at indexing time.
 2. Perform query evaluation in two phases:
 - Phase 1: Collect global stats
 - Phase 2: Execute query with stats

Trade-offs:

- More communication, but easier to retrofit
- Suitable for distributed systems
- Logical partitioning usually performs better in shared-memory setups

5. Term Partitioning in Inverted File Systems

- A single inverted file is built.
- Inverted lists are distributed across processors by term.

Query Evaluation:

- Broker sends each term to the processor storing its inverted list.
- Processors compute partial scores.
- Broker combines these into final results.

6. Indexing Alternatives: Suffix Arrays

6.1. Document Partitioning

- Each processor builds a local suffix array.
- During query:
 - Broker sends query to all
 - Each processor returns results
 - Broker merges results

6.2. Term Partitioning

- Suffix array is divided lexicographically.
- Each processor holds part of the array.
- All processors need access to the full text.

Challenge:

- Shared memory is needed for efficient access
- Not well-suited for distributed systems without shared memory

7. Signature Files

7.1. Document Partitioning

- Each processor builds signatures for its documents.
- Query signature is generated and broadcast.

- Each processor does local matching and returns results.

7.2. Term Partitioning

- Requires bit-sliced signature files
- Partitions the bit slices across processors

Limitation:

- High cost of merging and removing false positives
- Not recommended due to limited speedup

8. Performance and Implementation Notes

- Threads (e.g., Java threads, POSIX threads) are used for implementing parallel processes
- Provide abstractions for communication, concurrency, and synchronization.
- Enable scalable and maintainable parallel IR systems.

Feature	Logical Document Partitioning	Physical Document Partitioning	Term Partitioning
Index Sharing	Yes (shared)	No (independent)	Partial (term-based)
Ease of Update	Moderate	High (easier)	Difficult
Query Latency	Low	Moderate	Low
Communication	Low	High	Moderate
Best Use Case	Shared-memory systems	Distributed systems	Uniform query term distribution
Index Build Complexity	Moderate	Simple	High

9.2.3 SIMD Architectures

SIMD (Single Instruction, Multiple Data) architectures execute the same operation on multiple data points simultaneously. They're suitable for parallel processing tasks but are less versatile than MIMD architectures. A key example of SIMD is the **Thinking Machines CM-2**, which has been applied to both **signature file** and **inverted file** based IR algorithms.

- **Processing Element (PE):** Each PE in CM-2 has a 1-bit ALU and local memory.
- **Instruction Types:**
 - **Local Parallel Instructions:** Perform operations in parallel on each PE's local data.
 - **Non-local Instructions:** Involve inter-PE communication, e.g., finding a global max or vector sum.
- **Execution Model:** Controlled by a front-end host, which handles serial logic and sends parallel instructions to the PEs through a microcontroller.

2. Virtual Processors in CM-2

- CM-2 abstracts its physical processors into **virtual processors (VPs)**.
- A single physical processor handles multiple VPs sequentially.
- **Virtual Processing Ratio (VP):** Number of virtual processors per physical processor. As VP increases, execution time increases nearly linearly.

3. Signature Files on SIMD

Signature files are a natural fit for SIMD due to their binary and parallel nature.

3.1 Basic Search Process

1. Create a binary **query signature** from the query terms.
2. Compare it against every document's signature.
3. Mark documents with matching signatures as **potential matches**.
4. Optionally scan full text to **eliminate false positives (false drops)**.

3.2 probe.doc Routine

- **probe.doc** is the core subroutine that tests if a document signature matches a query term.
- It **uses hash functions to check bits** in the signature and returns a match indicator (1 or 0).
- **Parallel variables allow each VP to handle a separate document signature simultaneously.**

3.3 Segmented Signatures

If a document has more terms than can be encoded in one signature:

- **The document is divided into segments.**
- **Each segment gets its own signature.**
- **A match is declared if any segment matches the query.**

4. Boolean and Ranked Retrieval Using Signatures

4.1 Boolean Retrieval

- **Uses a recursive structure (**bquery.t**) for combining query terms with AND, OR, and NOT operators.**

- The final result is stored in a parallel Boolean variable that flags matching documents.

4.2 Ranked Retrieval

- Query terms have weights.
- Each match increases the score of a document by the term's weight.
- Final scores are stored in a parallel variable (**Doc.score**), and documents are ranked.

4.3 Ranking Techniques

- **Global ranking** sorts all scores (used for top-k hits).
- **Iterative global max** finds top-ranked documents in multiple passes (more efficient for small k).

5. Optimizing Signature File Access

5.1 Batch Processing

- If the entire signature file doesn't fit in memory, it is processed in **batches**.
- Multiple queries per batch can **amortize I/O costs**.
- Improves throughput but not response time.

5.2 Bit-Sliced Signature Files

Proposed by Panagopoulos and Faloutsos:

- Signature file is stored **column-wise** (bit-slices) instead of

row-wise (document-wise).

- Each bit-slice is a vertical column of the signature matrix.
- Only relevant bit-slices for the query are loaded.
- Each VP handles one document's bits across slices.
- Requires frame replacement policies and mapping hash functions to slices.

6. Inverted Files on SIMD

6.1 Basic Inverted File Layout

- **Postings Table:** Stores document IDs for each term.
- **Index:** Maps each term to a range in the postings table.
- Documents are scored using **accumulators (mailboxes)**.

6.2 Scoring Algorithm

- For each query term:
 - Look up postings in index.
 - Activate PEs that contain term postings.
 - Update document scores in accumulators.
- Communication is needed to pass weights between PEs, which adds overhead.

7. Partitioned Postings File

To reduce communication costs:

- Store each document's **postings and accumulator on the same processor**.
- Postings are loaded by column (not row), based on processor assignment.

- **Segmentation** ensures terms in each segment are lexically ordered.
- The **index** stores term tags and partition locations.

7.1 Advantages

- Reduces communication overhead.
- Supports fast scoring and sub-2-second query times on terabyte-scale data (CM-2 with 64K processors).

7.2 Space Overhead

- Approx. 33% space overhead.
- 10-20% may be due to padding in partitioned segments.

Summary

SIMD architectures like CM-2 are well-suited for:

- **Signature-based retrieval**, especially when false positives are acceptable or manageable.
- **Inverted file search**, though more complex, can be adapted using careful data layout and processor-aware partitioning.

Despite their limitations in flexibility compared to MIMD systems, SIMD architectures offer high-throughput parallel processing that can significantly improve IR performance on large datasets.

9.3 Distributed IR

9.3.1 Introduction

Distributed computing is the application of multiple computers connected by a network to solve a single problem.

In this model:

- Each computer (or node) may be independently owned and operated.
- A node can itself be a **parallel computer**, meaning distributed systems can nest parallelism.
- Nodes communicate using **network protocols** like TCP/IP, which is **slower** than shared memory communication in typical MIMD systems.
- This architecture provides **heterogeneity** and **flexibility**, supporting a public protocol/interface that all nodes adhere to.

Distributed IR Architecture Components

Distributed IR systems typically consist of:

- **Broker Process:**
 - Accepts client search requests.
 - Forwards requests to selected servers.
 - Collects intermediate results.
 - Combines them into a final result.

- **Server Processes:**
 - Each server resides on a separate node.
 - **Processes document subsets** or entire collections.
 - **Communicates with the broker over the network.**

This model supports **coarse-grained parallelism**, which is optimal when communication is expensive and computation per node is significant.

Document partitioning is highly suited to distributed IR because:

- **Documents can be divided into collections**, which naturally fit on separate servers.
- **Each server processes its self-contained subcollection**, reducing inter-server communication.
- The **workload is computationally intensive**, but **communication-light**, making it ideal for distributed systems.

By contrast, **term partitioning** is not well-suited because:

- It requires significant **inter-server communication** for merging partial results.
- It increases response latency due to **synchronization overhead**.

Engineering Issues in Distributed IR Systems

To implement a distributed IR system effectively, several engineering challenges must be addressed:

5.1. Search Protocol

The search protocol defines how clients and servers interact:

- Message syntax and semantics.
- Sequence of messages for:
 - Server discovery.
 - Query submission.
 - Result retrieval.
- Underlying transport (e.g., TCP/IP).

Minimum protocol capabilities include:

- Querying server metadata (e.g., list of searchable databases).
- Access to statistics for ranking or selection.
- Transmission of search queries and return of results.

5.2. Server Design

Each search server must:

- Accept and parse incoming requests.
- Spawn a thread or subprocess to handle each request.

- Apply **caching** to speed up repeated queries or term lookups.
- Optimize for **locality of reference** (e.g., accessing frequently used documents or terms faster).

5.3. Broker Design

The broker must:

- Accept user queries asynchronously.
- Select the relevant subset of servers (not all servers need to process every query).
- Distribute the query in parallel to selected servers.
- Merge intermediate results from servers into a single **ranked final list** for the user.

6. Algorithmic Issues in Distributed IR

Key algorithmic design questions include:

- **Document Distribution:**
 - Should documents be grouped by topic, source, or language?
 - Should collections be statically or dynamically partitioned?
- **Server Selection:**
 - Should all servers be queried, or only a few?

- Use server metadata (e.g., collection statistics) for intelligent selection.
- **Result Merging:**
 - How to combine partial rankings or hit-lists from each server.
 - May require normalization of scores due to differences in term distributions.

Examples of Search Protocols:

1. **Z39.50 Standard**

- A widely adopted **client/server IR protocol**.
- Supports **search, retrieval, and database discovery**.
- Suitable for **library systems and academic databases**.

2. **STARTS (Stanford Proposal for Internet Meta-Searching)**

- Designed for **distributed and heterogeneous search systems**.
- Developed to **address algorithmic challenges**, such as:
 - How to **merge results** from **different sources**.
 - How to **standardize scoring** and relevance.

- Supports advanced distributed IR features.

9.3.2 Collection Partitioning

Collection partitioning refers to how documents are assigned across different search servers in a distributed IR system. The strategy depends on administration, collection size, and search goals.

Types of Collection Partitioning

1. Independent / Heterogeneous Systems

- Documents are independently indexed and managed at each server.
- There's no central control over how documents are partitioned.
- Often, each server focuses on a specific topic, leading to semantic partitioning (e.g., legal, medical, or tech documents).
- Common in meta-search systems (like federated search engines accessing different services).

2. Centrally Administered Systems

When the system is centrally controlled, three partitioning strategies are typically used:

(a) Replicated Partitioning

- The entire collection is copied to every server.
- Used for high availability and load balancing.
- Indexing can be done by:
 - Having each server independently index the full replica.
 - Splitting documents for indexing, then merging partial indexes on each server.

⚠ Updates/deletions must be broadcast to all servers to maintain consistency.

(b) Random Distribution

- Documents are distributed randomly among servers.
- Each server gets a portion of the collection.
- Every query is broadcast to all servers, and results are merged.

Best when users expect to search the entire collection at once.

(c) Semantic Partitioning

- Documents are organized by topic or category (e.g., sports, medicine, finance).
- Done manually or using automatic clustering/categorization techniques.
- Improves search precision by sending queries to topic-relevant servers.

9.3.3 Source Selection

Source selection is the process of determining which of the distributed document collections are most likely to contain relevant documents for the current query, and therefore should receive the query for processing.

Source selection is the process of choosing which servers (or document collections) to search for a particular query.

Approaches to Source Selection

1. Broadcast to All Servers

- Simple but inefficient.
- Used when:
 - Document distribution is random.
 - Collections **overlap semantically**.
 - Communication costs are low.

2. Ranked Collection Selection

- Collections are **ranked** based on how likely they are to contain relevant results.
- Each collection is treated as a **"super-document"**.
- Query is compared against collection summaries using **cosine similarity** and **tf-idf** weighting.

Term weight formula:

- tf_i_j : frequency of term i in collection j
- $idf_i = \log(N / n_i)$
 - N = number of collections
 - n_i = number of collections containing term i

Problem: False Drops: A collection may rank highly overall, but no individual document inside may actually be relevant. Leads to unnecessary work.

Solution (Moffat & Zobel):

- Partition each collection into blocks (e.g., every 10 docs).
- Index blocks instead of whole collections.

- Helps balance **index size** vs. **accuracy**.

3. Content Modeling (Voorhees)

- Use **training queries** to build a **model** for each collection's content.
- When a new query comes in:
 - Match it to similar training queries.
 - Use model to **predict which collections to query** and how many results to expect.

9.3.4 Query Processing

Query processing in a distributed IR system is a multi-step procedure where **user queries are handled by multiple distributed servers to retrieve relevant documents efficiently.**

Steps in Query Processing

1. **Select Collections to Search**

- Choose relevant collections based on query content (unless using full broadcast).
- Techniques like source selection or semantic matching are used.

2. **Distribute the Query**

- **The broker sends the query to selected collections** using

a defined search protocol (e.g., TCP/IP, STARTS, Z39.50).

3. Evaluate Query in Parallel

- Each search server evaluates the query independently using its local index and algorithm.

4. Merge Results

- The broker collects and merges partial results into a final ranked list to return to the user.



Result Merging Strategies

The strategy for merging results depends on the query type and the structure of the document collections.

1. Boolean Queries

- Servers return Boolean result sets.
- Final result: union of all result sets.

2. Ranked Queries (Free-text Search)

Multiple approaches exist, ordered by complexity:

(a) Round-Robin Interleaving

- Simple interleaving of top results from each server.
- ✗ Drawback: Gives equal weight to results from all collections, regardless of relevance.

(b) Score-Based Merging

- Combine documents from each server by document relevance score.
- ✓ Works well if term statistics are globally consistent.
- ✗ Can fail when using semantic partitions or independent servers, as local statistics may not reflect global importance.

(c) Reranking with Collection Weights

- Proposed by Callan (139).
- Adjust scores based on collection relevance:

$$w_i = 1 + |C| * (\bar{s} - s_i) / \bar{s}$$

- $|C|$: number of collections
- s_i : score of collection i
- $\bar{s}$: average collection score
- Prioritizes results from more relevant collections.

3. Global Term Statistics-Based Merging

- Most accurate but complex.

- Two methods:

Using Existing Global Index

- If available (e.g., from source selection step), include global term statistics with the query.
- Servers adjust scoring accordingly.

Two-Round Communication

1. Round 1: Broker sends query, servers return collection statistics.
2. Round 2: Broker computes global statistics and resends for improved scoring.

4. Broker-Side Reranking

- Servers return term statistics per document.
- Broker reranks all documents centrally using a custom scoring algorithm.

- Produces results as if documents were indexed in a single collection.

9.3.5 Web Issues

There are **two approaches** for using distributed IR in Web search systems:

1. Centralized Web Search (Current Model)

- All crawled Web documents are stored and indexed centrally.
- **Distributed and parallel IR techniques** are used internally for scalability and performance.
- Examples: Google, Bing.

2. Truly Distributed Web IR (e.g., Harvest System)

- The Web's distributed nature is preserved.
- **Components:**
 - **Gatherers:** Collect content from various sources.
 - **Brokers:** Handle user queries and manage indexing and results.
 - **Broker Registry:** Helps users find the right broker for their topic.
- Suitable for **topic-specific search** systems or **federated search**.

Searching the Web

13.1 Introduction

The World Wide Web dates from the end of the 1980s and no one could have imagined its current impact. The boom in the use of the Web and its exponential growth are now well known. Just the amount of textual data available is estimated to be in the order of one terabyte. In addition, other media, such as images, audio, and video, are also available. Thus, the Web can be seen as a very large, unstructured but ubiquitous database. This triggers the need for efficient tools to manage, retrieve, and filter information from this database. This problem is also becoming important in large intranets, where we want to extract or infer new

information to support a decision process, a task called data mining.

We focus on text, because although there are techniques to search for images and other non-textual data, they cannot be applied (yet) on a large scale. We also emphasize syntactic search. That is, we search for Web documents that have user-specified words or patterns in their text, such words or patterns may or may not reflect the intrinsic semantics of the text. An alternative approach to syntactic search is to do a natural language analysis of the **text**. Although the techniques to preprocess natural language and extract the text semantics are not new, they are not yet very effective and they are also too costly for large amounts of data. In addition, in most cases they are only effective with well structured text, a thesaurus, and other contextual information.

There are basically three different forms of searching the Web. Two of them are well known and are frequently used. The first is to use search engines that index a portion of the Web documents as a **full-text database**. The second is to use Web directories, which classify selected Web documents by **subject**. The third and not yet fully available, is to search the Web exploiting its **hyperlink**.

13.2 Challenges

We can divide them in two classes: problems with the data itself and problems regarding the user and his interaction with the retrieval system.

The problems related to the data are

- **Distributed data:** due to the intrinsic nature of the Web, data spans over many computers and platforms. These computers are interconnected with no predefined topology and the available bandwidth and reliability on the network interconnections varies widely.
- **High percentage of volatile data:** due to Internet dynamics, new computers

and data can be added or removed easily (it is estimated that 40% of the Web changes every month). We also have dangling links and relocation problems when domain or file names change or disappear.

- **Large volume:** the exponential growth of the Web poses scaling issues that are difficult to cope with.
- **Unstructured and redundant data:** most people say that the Web is a distributed hypertext. However, this is not exactly so. Any hypertext has a conceptual model behind it, which is organized and adds consistency to the data and the hyperlinks. That is hardly true on the Web, even for individual documents. In addition, each HTML page is not well structured and some people use the term semi-structured *data*. Moreover, much Web data is repeated (mirrored or copied) or very similar. Approximately 30% of Web pages are (near) duplicates. Semantic redundancy can be even larger.
- **Quality of data:** the Web can be considered as a new publishing medium. However, there is, in most cases, no editorial process. So, data can be false, invalid (for example, because it is too old), poorly written or, typically, with many errors from different sources (typos, grammatical mistakes, OCR errors, etc.). Preliminary studies show that the number of words with typos can range from 1 in 200 for common words to 1 in 3 for foreign surnames .
- **Heterogeneous data:** in addition to having to deal with multiple media types and hence with multiple formats, we also have different languages and, what is worse, different alphabets, some of them very large (for example, Chinese or Japanese Kanji).

Most of these problems (such as the variety of data types and poor data quality) are not solvable simply by software improvements. In fact, many of them will not change (and they should not, as in the case of language diversity!) because they are problems (also features) intrinsic to human nature.

The second class of problems are those faced by the user during the

interaction with the retrieval system. There are basically two problems:

(1) how to specify a query and (2) how to interpret the answer provided by the system.

So, the overall challenge, in spite of the intrinsic problems posed by the Web, is to submit a good query to the search system, and obtain a manageable and relevant answer. Moreover, in practice we should try to achieve the latter goal even for poorly formulated queries.

13.3 Search Engines

Search engines are specialized Information Retrieval (IR) systems designed to manage and search through massive amounts of web data. Unlike traditional IR systems, search engines operate under strict constraints and massive scale, requiring specialized architectures and indexing strategies. web search engines cannot access the full text of documents at query time. They rely entirely on precomputed indexes due to: The impracticality of storing local copies of the entire web, The latency of retrieving remote documents in real-time.

13.3.1 Centralized Architecture

Most search engines use a centralized crawler-indexer architecture. Crawlers are programs (software agents) that traverse the Web sending new or updated pages to a main server where they are indexed. Crawlers are also called robots, spiders, wanderers, walkers, and knowbots. In spite of their name, a crawler does not actually move to and run on remote machines, rather the crawler runs on a local system and sends requests to remote Web servers. The index is used in a centralized fashion to answer queries submitted from different places in the Web.

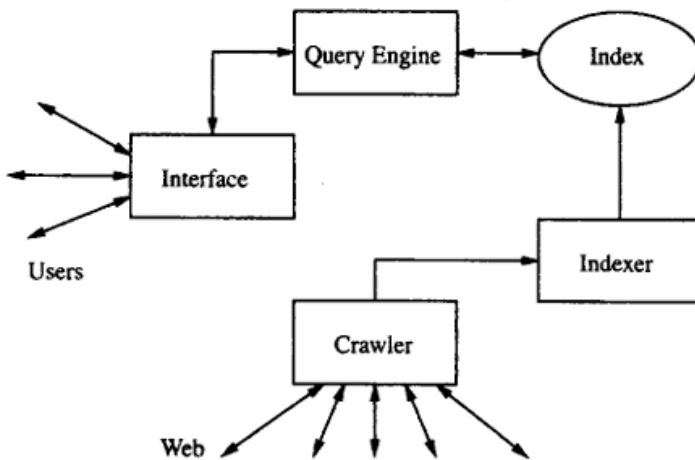


Figure 13.3 Typical crawler-indexer architecture.

Figure 13.3 shows the software architecture of a search engine based on the AltaVista architecture. It has two parts: one that deals with the users, consisting of the user interface and the query engine and another that consists of the crawler and indexes modules. In 1998, the overall AltaVista system was running on 20 multi-processor machines, all of them having more than 130 Gb of RAM and over 500 Gb of disk space. Only the query engine uses more than 75% of these resources.

The main problem faced by this architecture is the gathering of the data, because of the highly dynamic nature of the Web, the saturated communication links, and the high load at Web servers. Another important problem is the volume of the data. In fact, the crawler-indexer architecture may not be able to cope with Web growth in the near future. Particularly important is good load balancing between the different activities of a search engine, internally (answering queries and indexing) and externally (crawling).

The largest search engines, considering Web coverage in June 1998, were AltaVista , HotBot , Northern Light , and Excite , in that order.

Search engine	URL	Web pages indexed
AltaVista	www.altavista.com	140
AOL Netfind	www.aol.com/netfind/	–
Excite	www.excite.com	55
Google	google.stanford.edu	25
GoTo	goto.com	–
HotBot	www.hotbot.com	110
Infoseek	www.infoseek.com	30
Lycos	www.lycos.com	30
Magellan	www.mckinley.com	55
Microsoft	search.msn.com	–
NorthernLight	www.nlsearch.com	67
WebCrawler	www.webcrawler.com	2

Table 13.2 URLs and estimated size (millions) of the largest search engines (May 1998).

Most search engines are based in the United States and focus on documents in English. Nevertheless, there are search engines specialized in different countries and/or languages, which are able, for instance, to query and retrieve documents written in Kanji (Chinese, Japanese, and Korean). Also there are search engines that take other approaches, like Ask Jeeves! which simulates an interview or DirectHit which ranks the Web pages in the answer in order of their popularity. We should also mention those search engines aimed at specific topics, for example the Search Broker which allows us to search in many specific topics and DejaNews which searches the USENET archives.

13.3.2 Distributed Architecture

There are several variants of the crawler-indexer architecture. Among them, the most important is Harvest. Harvest uses a distributed architecture to gather and distribute data, which is more efficient than the crawler architecture. The main drawback is that Harvest requires the coordination of several Web servers.

The Harvest distributed approach addresses several of the problems of the crawler-indexer architecture, such as: (1) Web servers receive requests from

different crawlers, increasing their load; (2) Web traffic increases because crawlers retrieve entire objects, but most of their content is discarded; and (3) information is gathered independently by each crawler, without coordination between all the search engines.

To solve these problems, Harvest introduces two main elements: gatherers and brokers. A gatherer collects and extracts indexing information from one or more Web servers. Gathering times are defined by the system and are periodic (i.e. there are harvesting times as the name of the system suggests). A broker provides the indexing mechanism and the query interface to the data gathered. Brokers retrieve information from one or more gatherers or other brokers, updating incrementally their indices. Depending on the configuration of gatherers and brokers, different improvements on server load and network traffic are achieved.

For example, a gatherer can run on a Web server, generating no external traffic for that server. Also, a gatherer can send information to several brokers, avoiding work repetition. Brokers can also filter information and send it to other brokers. This design allows the sharing of work and information in a very flexible and generic manner. An example of the Harvest architecture is shown in Figure 13.4

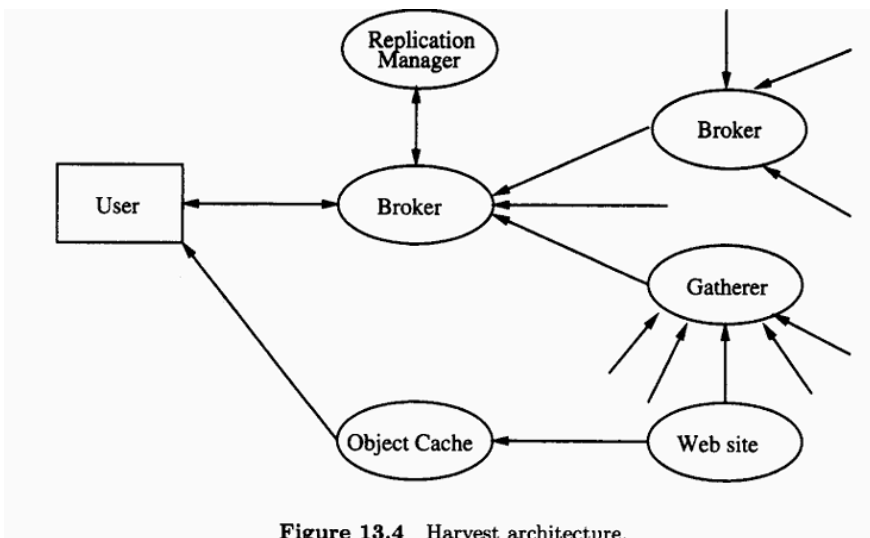


Figure 13.4 Harvest architecture.

One of the goals of Harvest is to build topic-specific brokers, focusing the index contents and avoiding many of the vocabulary and scaling problems of generic indices. Harvest includes a distinguished broker that allows other brokers to register information about gatherers and brokers. This is useful to search for an appropriate broker or gatherer when building a new system.

The Harvest architecture also provides replicators and object caches. A replicator can be used to replicate servers, enhancing user-base scalability. For example, the registration broker can be replicated in different geographic regions to allow faster access. Replication can also be used to divide the gathering process between many Web servers. Finally, the object cache reduces network and server load, as well as response latency when accessing Web pages.

Currently, there are hundreds of Harvest applications on the Web (for example, the CIA, NASA, the US National Academy of Sciences, and the US Government Printing Office), as this software is on the public domain. Netscape's Catalog Server is a commercial version of Harvest and Network Appliances' cache is a commercial version of the Harvest Cache.

13.3.3 User Interfaces

There are two important aspects of the user interface of search engines: the query interface and the answer interface. The basic query interface is a box where one or more words can be typed. For example, in AltaVista a sequence of words is a reference to the union of all the Web pages having at least one of those words, while in HotBot it is a reference to the Web pages having all the words. Another problem is that the logical view of the text is not known, that is, some search engines use stopwords, some do stemming, and some are not case sensitive.

All search engines also provide a query interface for complex queries as well as a command language including Boolean operators and other features, such as phrase search, proximity search, and wild cards. They provide several filtering functions. The results can be filtered by additional words that must be present or

absent from the answer or in a particular field such as the URL or title, language, geographic region or Internet domain, date range, or inclusion of specific data types such as images or audio.

The answer usually consists of a list of the ten top ranked Web pages. Each entry in this list includes some information about the document it represents. Typically, the information includes the URL, size, the date when the page was indexed, and a couple of lines with its content (title plus first lines or selected headings or sentences). Some search engines allow the user to change the number of pages returned in the list and the amount of information per page, but in most cases this is fixed or limited to a few choices. The order of the list is typically by relevance, but sorting by URL or date is also available in some engines. In addition, most search engines also have an option to find documents similar to each Web page in the answer.

The user can also refine the query by constructing more complex queries based on the previous answer. The Web pages retrieved by the search engine in response to a user query are ranked, usually using statistics related to the terms in the query. In some cases this may not have any meaning, because relevance is not fully correlated with statistics about term occurrence within the collection. Some search engines also taking into account terms included in metatags or the title, or the popularity of a Web page to improve the ranking.

13.3.4 Ranking

Most search engines use variations of the Boolean or vector model to do ranking. Yuwono and Lee propose three ranking algorithms in addition to the classical tf-idf scheme. They are called Boolean spread, vector spread, and most-cited. The first two are the normal ranking algorithms of the Boolean and vector model extended to include pages pointed to by a page in the answer or pages that point to a page in the answer. The third, most-cited, is based only on the terms included in pages having a link to the pages in the answer.

A comparison of these techniques considering 56 queries over a collection of 2400 Web pages indicates that the vector model yields a better recall-precision curve, with an average precision of 75%.

Some of the new ranking algorithms also use hyperlink information. This is an important difference between the Web and normal IR databases. The number of hyperlinks that point to a page provides a measure of its popularity and quality. Also, many links in common between pages or pages referenced by the same page often indicates a relationship between those pages.

Three examples of ranking techniques that exploit these facts, but they differ in that two of them depend on the query and the last does not.

The first is WebQuery which also allows visual browsing of Web pages. WebQuery takes a set of Web pages (for example, the answer to a query) and ranks them based on how connected each Web page is. Additionally, it extends the set by finding Web page that are highly connected to the original set.

A better idea is due to Kleinberg and used in HITS (Hypertext Induced Topic Search). This ranking scheme depends on the query and considers the set of pages S that point to or are pointed by pages in the answer. Pages that have many links pointing to them in S are called authorities (that is, they should have relevant content). Pages that have many outgoing links are called hubs (they should point to similar content).

A positive two-way feedback exists: better authority pages come from incoming edges from good hubs and better hub pages come from outgoing edges to good authorities. Let $H(p)$ and $A(p)$ be the hub and authority value of page p . These values are defined such that the following equations are satisfied for all pages p :

$$H(p) = \sum_{u \in S \mid p \rightarrow u} A(u), \quad A(p) = \sum_{v \in S \mid v \rightarrow p} H(v)$$

where $H(p)$ and $A(p)$ for all pages are normalized (in the original paper, the sum of the squares of each measure is set to one). These values can be determined

through an iterative algorithm, and they converge to the **principal** eigenvector of the link matrix of S . In the case of the Web, to avoid an explosion of the size of S , a maximal number of pages pointing to the answer can be defined. This technique does not work with non-existent, repeated, or automatically generated links. One solution is to weight each link based on the surrounding content. A second problem is that the topic of the result can become diffused.

The last example is **PageRank**, which is part of the ranking algorithm used by Google. PageRank simulates a user navigating randomly in the Web who jumps to a random page with probability q or follows a random hyperlink (on the current page) with probability $1 - q$. It is further assumed that this user never goes back to a previously visited page following an already traversed hyperlink backwards. This process can be modeled with a Markov chain, from where the stationary probability of being in each page can be computed. Let $C(e)$ be the number of outgoing links of page e and suppose that page o is pointed to by pages p_i to p_q . Then, the PageRank, $PR(a)$ of e is defined as

$$PR(a) = q + (1 - q) \sum_{i=1}^n PR(p_i) / C(p_i)$$

where q must be set by the system (a typical value is 0.15). Notice that the ranking (weight) of other pages is normalized by the number of links in the page. PageRank can be computed using an iterative algorithm, and corresponds to the principal eigenvector of the normalized link matrix of the Web (which is the transition matrix of the Markov chain)

13.3.5 Crawling the Web

Crawling is the process of discovering and downloading web content.

Crawling Methods

- Start from a **set of seed URLs**.
- Extract more URLs recursively using:

- **Breadth-first traversal:** Wide and shallow; fetches all immediate links before going deeper.
- **Depth-first traversal:** Deep and narrow; follows links down to depth before backtracking.

Crawl Management

- **Challenges:**
 - Avoid fetching the same page multiple times.
 - Respect bandwidth and server limits.
 - Handle page updates efficiently.
- **Solutions:**
 - Assign crawlers by **domain** or **geographic region**.
 - Maintain a **crawl history**.
 - Use **robots.txt** to honor crawl restrictions.

Crawl Freshness and Frequency

- Crawled pages can be **1 day to 2 months old**.
- Popular pages or those with **high update frequency** are crawled more often.
- Some **crawlers learn** update frequencies to optimize revisits.

Technical Issues

- Robots must **avoid overwhelming servers**.
- **Some pages** (like those using frames or requiring authentication) are hard to crawl.
- Percentage of **broken links** in search engine indices: **2–9%**.

13.3.6 Indices

An **index** is a data structure that allows fast retrieval of documents matching user queries.

Inverted File Index: Main technique used. Contains: A **sorted vocabulary** (unique words), For each word, a **posting list** of documents where it appears.

Index Compression and Storage

- To reduce size:
 - Use **stopword removal, stemming, normalization.**
 - Compress posting lists.
- Index size:
 - ~30% of original text (can go down to 10% with compression).
 - For 100 million pages, around **150 GB is needed.**

Additional **Metadata Includes:** URL, Document title, Snippets (first few lines/headings), Creation date and size, Helps display relevant info to users.

Query Processing

- Queries are answered by **binary search** on the vocabulary.
- Multi-word queries require merging of posting lists.

Phrase and Proximity Queries

- Requires storing **word positions** in documents.
- Used in **phrase search** and **nearby word** matching.
- Full position indexing is space-consuming.

Wildcards and Approximate Search

- Prefix queries use two binary searches.
- Complex patterns (like wildcards) use **sequential scan** over the vocabulary.

Block-based Indexing

Reduces index size by **indexing logical blocks** instead of full documents. Used in systems like **Glimpse** and **Harvest**. Not widely used in centralized search engines due to network constraints.

13.4 Browsing

Web tools which are based on browsing and searching, in particular **Web directories**. Although the Web coverage provided by directories is very low (less than 1% of all Web pages), the answers returned to the user are usually much more relevant.

13.4.1 Web Directories

The best and oldest example of a Web directory is **Yahoo!**, which is likely the most used searching tool. Other large Web directories include **eBLAST**, **LookSmart**, **Magellan**, and **NewHoo**. Some of them are hybrids, because they also provide searches on the whole Web. Most search engines also provide subject categories nowadays, including AltaVista Categories, AOL Netfind, Excite Channels, HotBot, Infoseek, **Lycos** Subjects, and WebCrawler Select. are specific to some areas.

For example, there are Web sites focused on business, news, and, in particular, research bibliography. Web directories are also called catalogs, yellow pages, or subject directories.

Web directory	URL	Web sites	Categories
eBLAST	www.eblast.com	125	–
LookSmart	www.looksmart.com	300	24
Lycos Subjects	a2z.lycos.com	50	–
Magellan	www.mckinley.com	60	–
NewHoo	www.newhoo.com	100	23
Netscape	www.netscape.com	–	–
Search.com	www.search.com	–	–
Snap	www.snap.com	–	–
Yahoo!	www.yahoo.com	750	–

Table 13.3 URLs, Web pages indexed and categories (both in thousands) of some Web directories (beginning of 1998).

Arts & Humanities	Local
Automotive	News
Business & Economy	Oddities
Computers & Internet	People
Education	Philosophy & Religion
Employment	Politics
Entertainment & Leisure	Recreation
Games	Reference
Government	Regional
Health & Fitness	Science & Technology
Hobbies & Interests	Shopping & Services
Home	Social Science
Investing	Society & Culture
Kids & Family	Sports
Life & Style	Travel & Tourism
Living	World

Table 13.4 The first level categories in Web directories.

Directories are hierarchical taxonomies that classify human knowledge.

The largest directory, Yahoo!, has close to one million pages classified, followed by LookSmart, which has about 24,000 categories in total. Yahoo! also offers 14 regional or country specialized directories in other languages including Chinese, Danish, French, German, Italian, Japanese, Korean, Norwegian, Spanish, and Swedish. In most cases, pages have to be submitted to the Web directory, where they are reviewed, and, if accepted, classified in one or more categories of the hierarchy. Although the taxonomy can be seen as a tree, there are cross references, so it is really a directed acyclic graph.

The main advantage of this technique is that if we find what we are looking for, the answer will be useful in most cases. On the other hand, the main disadvantage is that the classification is not specialized enough and that not all Web

pages are classified. The last problem becomes worse every day as the Web grows. The efforts to do automatic classification, by using clustering or other techniques, are very old. However, up to now, natural language processing is not 100% effective in extracting relevant terms from a document. Thus, classification is done manually by a limited number of people. This is a potential problem with users having a different notion of categories than the manmade categorization.

Web directories also allow the user to perform a search on the taxonomy descriptors or in the Web pages pointed to by the taxonomy. In fact, as the number of classified Web pages is small, we can even afford to have a copy of all pages. In that case they must be updated frequently, which may pose performance and temporal validity problems. In addition, most Web directories also send the query to a search engine (through a strategic alliance) and allow the whole Web to be searched.

13.4.2 Combining Searching with Browsing

Usually, users either browse following hypertext links or they search a Web site (or the whole Web). Currently, in Web directories, a search can be reduced to a subtree of the taxonomy. However, the search may miss related pages that are not in that part of the taxonomy.

WebGlimpse is a tool that tries to solve these problems by combining browsing with searching. WebGlimpse attaches a small search box to the bottom of every HTML page, and allows the search to cover the neighborhood of that page or the whole site, without having to stop browsing. This is equivalent to following hypertext links that are constructed on the fly through a neighborhood search. WebGlimpse can be useful in building indices for personal Web pages or collections of favorite URLs.

First, WebGlimpse indexes a Web site (or a collection of specific documents) and computes neighborhoods according to user specifications. As a result,

WebGlimpse adds the search boxes to selected pages, collects remote pages that are relevant, and caches those pages locally. Later, the users can search in the neighborhood of a page using the search boxes. As the name suggests, WebGlimpse uses Glimpse as its search engine.

The neighborhood of a Web page is defined as the set of Web pages that are reachable by a path of hypertext links within a maximum predefined distance. This distance can be set differently for local and remote pages.

13.5 Metasearchers

Metasearchers are Web servers that send a given query to several search engines, Web directories and other databases, collect the answers and unify them. Examples are Metacrawler and SavvySearch.

The main advantages of metasearchers are the ability to combine the results of many sources and the fact that the user can pose the same query to various sources through a single common interface. Metasearchers differ from each other in how ranking is performed in the unified result (in some cases no ranking is done), and how well they translate the user query to the specific query language of each search engine or Web directory (the query language common to all of them could be small).

Metasearchers can also run on the client, for example, Copernic, EchoSearch, WebFerret, WebCompass, and WebSeeker. There are others that search several sources and show the different answers in separate windows, such as All4One, OneSeek, Proteus, and Search Spaniel.

The advantages of metasearchers are that the results can be sorted by different attributes such as host, keyword, date, etc; which can be more

informative than the output of a single search engine. Therefore browsing the results should be simpler. On the other hand, the result is not necessarily all the Web pages matching the query, as the number of results per search engine retrieved by the metasearcher is limited (it can be changed by the user, but there is an upper limit). Nevertheless, pages returned by more than one search engine should be more relevant.

Metasearcher	URL	Sources used
Cyber 411	www.cyber411.com	14
Dogpile	www.dogpile.com	25
Highway61	www.highway61.com	5
Inference Find	www.infind.com	6
Mamma	www.mamma.com	7
MetaCrawler	www.metacrawler.com	7
MetaFind	www.metafind.com	7
MetaMiner	www.miner.uol.com.br	13
MetaSearch	www.metasearch.com	–
SavvySearch	savvy.cs.colostate.edu :2000	>13

Table 13.5 URLs of metasearchers and number of sources that they use (October 1998).

We expect that new metasearchers will do better ranking. A first step in this direction is the **NEC Research Institute metasearch engine, Inquirus**. The main difference is that Inquirus actually downloads and analyzes each Web page obtained and then displays each page, highlighting the places where the query terms were found. The results are displayed as soon as they are available in a progressive manner, otherwise the waiting time would be too long. This technique also allows non-existent pages or pages that have changed and do not contain the query any more to be discarded, and, more important, provides for better ranking than normal search engines. On the other hand, this metasearcher is **not available to the general public**.

Measure	Average value	Range
Number of words	2.35	0 to 393
Number of operators	0.41	0 to 958
Repetitions of each query	3.97	1–1.5 million
Queries per user session	2.02	1–173,325
Screens per query	1.39	1–78,496

Table 13.6 Queries on the Web: average values.

13.6 Finding the Needle in the Haystack

User Problems

Some of the problems faced by the user when interacting with the query interfaces currently provided by search engines.

First, the user does not exactly understand the meaning of searching using a set of words. Second, the user may get unexpected answers because he is not aware of the logical view of the text adopted by the system. An example is the use of uppercase letters when the search engine is not case sensitive. Hence, a word like 'Bank' loses part of its semantics if we search for 'bank.' Simple experiments also show that due to typos or variations of a word, even if correctly capitalized, 10—20% of the matches can be lost. Similarly, foreign names or words that are difficult to spell may appear incorrectly which may result in a loss of up to 50% of the relevant answers. Another problem is that most users have trouble with Boolean logic. In natural language, sometimes we use 'and' and 'or' with different meanings depending on the context. For example, when choosing between two things, we use an exclusive 'or, which does not match the Boolean interpretation. Because of this, several studies show that around 80% of the queries do not use any Boolean or other operation. For these reasons many people have trouble using command query languages, and query forms should clearly specify which words must or must not be contained in a document that belongs to the answer.

Users select a search engine mainly based on ease of use, speed, coverage, relevance of the answer, and habit. The main purposes are research, leisure, business, and education. The main problems found are that novice users do not know how to start and lack the general knowledge that would help in finding better answers. Other problems are that search engines are slow, that the answer is too large, not very relevant, and not always up to date. Also, most people do not care about advertising, which is one of the main sources of funding for search engines. When searching, 25% of the users use a single keyword, and on average their

queries have only two or three terms. In addition, about 15% of the users restrict the search to a predefined topic and most of them (nearly 80%) do not modify the query. In addition, most users (about 85%) only look at the first screen with results and 64% of the queries are unique. Also, many words appear in the same sentence, suggesting that proximity search should be used. There are also studies about users' demographics and software and hardware used.

13.5.1 Teaching the User

Interfaces are slowly improving in assisting the user with the task of acquiring a better grasp of what Web pages are being retrieved. Query forms must specify clearly if one or all the words must be in a page, which words should not be in a page, etc., without using a written Boolean query language. Second, users should try to give as many terms as possible, in particular terms that must be or should not be in the pages. In particular, a user should include all possible synonyms of a word. If the user can restrict the search to a field (for example, the page title) or limit some attribute (date, country), this will certainly reduce the size of the answer. In case of doubt, the user should remember to look at the help information provided by the search engine. If he cannot find where one of the relevant terms is in a page, he can use the Find option of the browser.

Even if we are able to pose a good query, the answer can still be quite large. Considering that the visual tools mentioned before are not yet available for the general public, the user must learn from experience. There are many strategies for quickly finding relevant answers. If the user is looking for an institution, he can always try to guess the corresponding URL by using the wvu prefix followed by a guessed institution acronym or brief name and ending with a top level domain (country code or con, edu, org, gov for the US). If this does not work, the user can search the institution name in a Web directory.

If we are looking for work related to a specific topic, a possible strategy is:

- (1) select an article relevant to the topic, if possible with non-common author

surnames or title keywords (if it is not available, try any bibliographic database or a Web directory search for a first reference); and (2) use a search engine to find all Web pages that have all those surnames and keywords. Many of the results are likely to be relevant, because we can find: (a) newer papers that reference the initial reference, (b) personal Web pages of the authors, and most important, (c) pages about the topic that already contain many relevant references. This strategy can be iterated by changing the reference used as better references appear during the search.

Web poses so many problems that it is easier and more effective to teach the user how to properly profit from search engines and Web directories, rather than trying to guess what the user really wants. Given that the coverage of the search engines is low, use several engines or a metasearcher. Also, remember that you have to evaluate the quality of each answer, even if it appears to be relevant. Remember that anybody can publish in the Web, and that does not mean that the data is correct or still valid.

The lessons learned in the examples shown above are: (1) search engines still return too much hay together with the needle; and (2) Web directories do not have enough depth to find the needle. So, we can use the following rules of thumb: **Specific queries** Look in an encyclopedia, that is the reason that they exist. In other words, do not forget libraries.

- **Broad queries** Use Web directories to find good starting points.
- **Vague queries** Use Web search engines and improve the query formulation based on relevant answers.

13.6 Searching using Hyperlinks

In this section we cover other paradigms to search the Web, which are based on exploiting its hyperlinks. They include Web query languages and dynamic searching. These ideas are still not widely used due to several reasons, including

performance limitations and lack of commercial products.

Web Query Languages

Up to this point, queries have been based on the content of each page. However, queries can also include the link structure connecting Web pages. For example, we would like to search for all the Web pages that contain at least one image and are reachable from a given site following at most three links. To be able to pose this type of query, different data models have been used. The most important are a labeled graph model to represent Web pages (nodes) and hyperlinks (edges) between Web pages, and a semi-structured data model to represent the content of Web pages. In the latter model, the data schema is not usually known, may change over time, may be large and descriptive, etc.

Although some models and languages for querying hypertext were proposed before the Web appeared, the first generation of Web query languages were aimed at combining content with **structure**. These languages combine patterns that appear within the documents with graph queries describing link structure (using path regular expressions). They include W3QL, WebSQL, WebLog, and WQL. The second generation of languages, called Web data manipulation languages, maintain the emphasis on semi-structured data. However, they extend the previous languages by providing access to the structure of Web pages (the model also includes the internal structure) and by allowing the creation of new structures as a result of a query. Languages in this category include **STRUQL**, **FLORID** and **WebOQL**. All the languages mentioned are meant to be used by programs, not final users. Web query languages have been extended to other Web tasks, such as extracting and integrating information from Web pages, and constructing and restructuring Web sites.

Dynamic Search and Software Agents

Dynamic search in the Web is equivalent to sequential text searching. The idea is to use an online search to discover relevant information by following links. The main advantage is that you are searching in the current structure of the Web, and not in what

is stored in the index of a search engine. While this approach is slow for the entire Web, it might be used in small and dynamic subsets of the Web. The first heuristic devised was the *fish search*, which exploits the intuition that relevant documents often have neighbors that are relevant. Hence, the search is guided by following links in relevant documents. This was improved by *shark search*, which does a better relevance assessment of neighboring pages. The main idea of these algorithms is to follow links in some priority, starting from a single page and a given query. At each step, the page with highest priority is analyzed. If it is found to be relevant, a heuristic decides to follow or not to follow the links on that page. If so, new pages are added to the priority list in the appropriate positions.

Related work includes software agents for searching specific information on the Web. This implies dealing with heterogeneous sources of information which have to be combined. Important issues in this case are how to determine relevant sources and how to merge the results retrieved (the fusion problem). Examples are shopping robots such as Jango, Junglee, and Express.